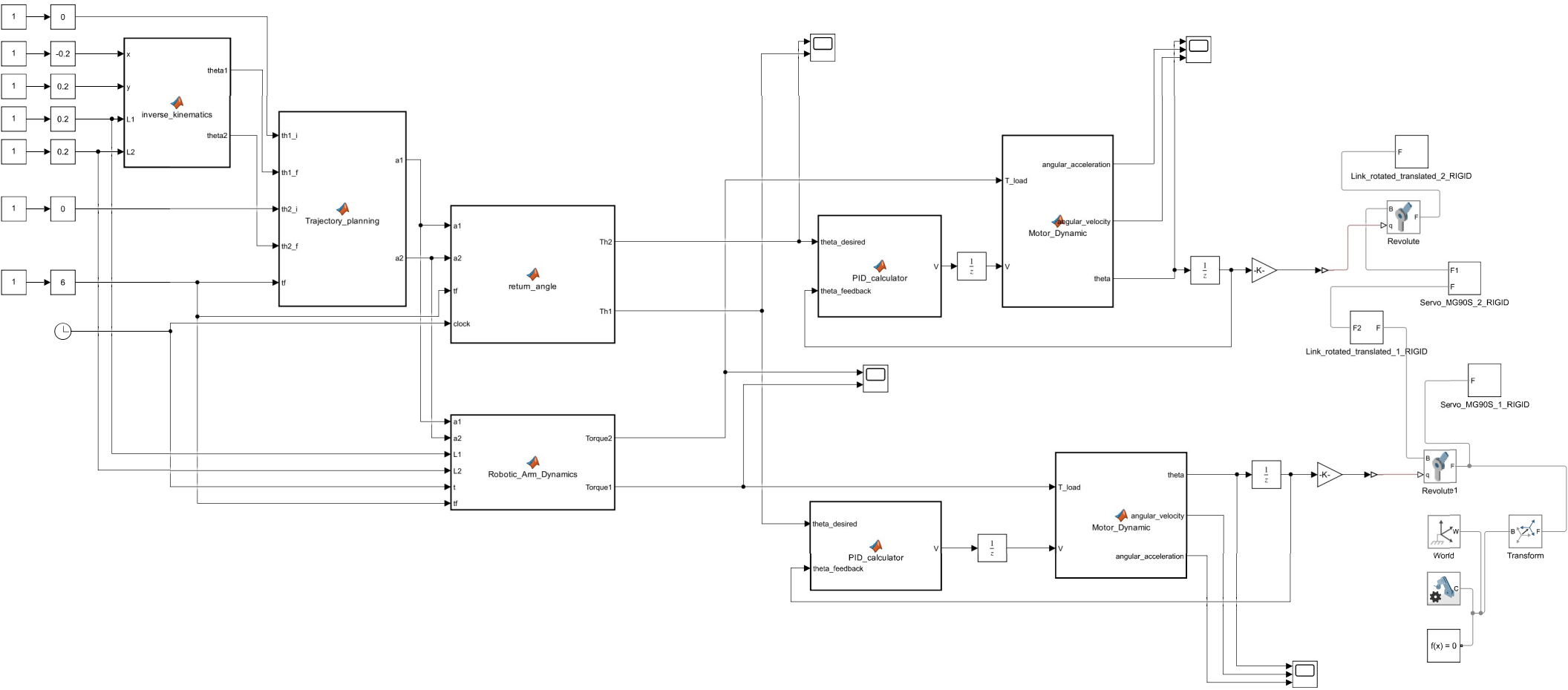
1
0
-0.2
0.2
6
x
y
L1
L2
theta1
theta2
inverse_kinematics
th1_i
th1_f
th2_i
th2_f
tf
a1
a2
Trajectory_planning
clock
Th2
Th1
return_angle
t
Torque2
Torque1
Robotic_Arm_Dynamics
theta_desired
theta_feedback
PID_calculator
V
T_load
angular_acceleration
angular_velocity
theta
Motor_Dynamic
-K-
Link_rotated_translated_2_RIGID
Revolute
F1
F
Servo_MG90S_2_RIGID
F2
Link_rotated_translated_1_RIGID
Servo_MG90S_1_RIGID
Revolute1
World
Transform
f(x) = 0

```
function [theta1, theta2]= inverse_kinematics(x,y, L1, L2)
% this function takes The final desired coordinates of the RA
% as inputs and generates required angles to achieve them as output

    r_squared = x^2 + y^2;

    % Check reachability
    if sqrt(r_squared) > (L1 + L2) || sqrt(r_squared) < abs(L1 - L2)
        error('Target is out of reach for the given arm dimensions.');
    end

    % Compute theta2
    cos_theta2 = (r_squared - L1^2 - L2^2) / (2 * L1 * L2);
    theta2 = acos(cos_theta2);  % Elbow-down solution

    % Compute theta1
    k1 = L1 + L2 * cos(theta2);
    k2 = L2 * sin(theta2);
    theta1 =  (atan2(k2, k1) - atan2(y, x));

    % Optional: return angles in degrees
    theta1 = rad2deg(theta1);
    theta2 = rad2deg(theta2);
    fprintf("Calculated angle %.2f, %.2f",theta1, theta2 );
end
```

```
function [Torque2, Torque1] = Robotic_Arm_Dynamics(a1, a2, L1, L2, t, tf)

    if t < 0
        t = 0;
    elseif t > tf
        t = tf;
    end


    g = 9.81;
    m1= 0.1;
    m2= 0.1;
    % Center of mass at L/2
    lc1 = L1 / 2;
    lc2 = L2 / 2;

    % Moments of inertia (rod about center)
    % I1 = (1/12) * m1 * L1^2;
    % I2 = (1/12) * m2 * L2^2;

    % Symbolic expressions for theta, theta_dot, theta_ddot
    th1 = a1(1) + a1(2)*t + a1(3)*t^2 + a1(4)*t^3;
    th2 = a2(1) + a2(2)*t + a2(3)*t^2 + a2(4)*t^3;

    th1dot =  a1(2) + 2*a1(3)*t + 3*a1(4)*t^2;
    th2dot =  a2(2) + 2*a2(3)*t + 3*a2(4)*t^2;

    th1ddot =   2*a1(3) + 2*3*a1(4)*t;
    th2ddot =   2*a2(3) + 2*3*a2(4)*t;

    %% Position of center of mass
    % x1 = lc1 * cos(th1);
    % y1 = lc1 * sin(th1);
    %
    % x2 = L1 * cos(th1) + lc2 * cos(th1 + th2);
    % y2 = L1 * sin(th1) + lc2 * sin(th1 + th2);





    d_dhoL_dho_th1dot_dt = lc1^2*m1*th1ddot + lc1^2*m2*th1ddot + lc2^2*m2*th1ddot + lc2^2*m2*th2ddot + 2*lc1*lc2*m2*cos(th2)*th1ddot
    d_dhoL_dho_th2dot_dt = lc2^2*m2*th1ddot + lc2^2*m2*th2ddot + lc1*lc2*m2*cos(th2)*th1ddot - lc1*lc2*m2*sin(th2)*th1dot*th2dot;

    dhoL_dho_th1 = - g*m2*(lc2*cos(th1 + th2) + lc1*cos(th1)) - lc1*g*m1*cos(th1);
    dhoL_dho_th2 = - (m2*(2*lc2*sin(th1 + th2)*(lc1*th1dot*cos(th1) + lc2*cos(th1 + th2)*(th1dot + th2dot))*(th1dot + th2dot) - 2*lc

    Torque1 = d_dhoL_dho_th1dot_dt - dhoL_dho_th1
    Torque2 = d_dhoL_dho_th2dot_dt - dhoL_dho_th2

end
```

```
function [ a1 , a2] = Trajectory_planning(th1_i, th1_f, th2_i, th2_f, tf)
%     coder.extrinsic('syms', 'solve', 'subs');
% syms t c0 c1 c2 c3 real
%
%     % Declare symbolic variables
%     t = syms('t');
%     c0 = syms('c0');
%     c1 = syms('c1');
%     c2 = syms('c2');
%     c3 = syms('c3');
%
%     % Time interval
%     t0 = 0;
%     tf = time_span;
%
%     % Define cubic polynomial and its derivative
%     theta = c0 + c1*t + c2*t^2 + c3*t^3;
%     thetadot = diff(theta, t);
%
%     % Initial and final angular velocities
%     omega1_i = 0; omega1_f = 0;
%     omega2_i = 0; omega2_f = 0;
%
%     %% Joint 1 trajectory equations
%     eqns_1 = [
%         subs(theta, t, t0) == th1_i;
%         subs(thetadot, t, t0) == omega1_i;
%         subs(theta, t, tf) == th1_f;
%         subs(thetadot, t, tf) == omega1_f;
%     ];
%
%     S1 = solve(eqns_1, [c0, c1, c2, c3]);
%
%     % Substitute solution into theta for joint 1
%     omega1 = simplify(subs(theta, [c0, c1, c2, c3], [S1.c0, S1.c1, S1.c2, S1.c3]));
%
%     %% Joint 2 trajectory equations
%     eqns_2 = [
%         subs(theta, t, t0) == th2_i;
%         subs(thetadot, t, t0) == omega2_i;
%         subs(theta, t, tf) == th2_f;
%         subs(thetadot, t, tf) == omega2_f;
%     ];
%
%     S2 = solve(eqns_2, [c0, c1, c2, c3]);
%
%     % Substitute solution into theta for joint 2
%     omega2 = simplify(subs(theta, [c0, c1, c2, c3], [S2.c0, S2.c1, S2.c2, S2.c3]));

    % function [theta1, theta2] = Trajectory_planning(th1_i, th1_f, th2_i, th2_f, tf, t)
% Cubic trajectory planner for two joints
% Inputs:
%   th1_i, th1_f: Initial and final angle for joint 1
%   th2_i, th2_f: Initial and final angle for joint 2
%   tf: total trajectory time
%   t: current time

    % Clamp time to [0, tf] for safety
    % if t < 0
    %     t = 0;
    % elseif t > tf
```

```
    %     t = tf;
    % end

    % Boundary velocities
    v0 = 0;
    vf = 0;

    % Time matrix
    T = [1 0 0^2 0^3;
         0 1 2*0 3*0^2;
         1 tf tf^2 tf^3;
         0 1 2*tf 3*tf^2];

    % Solve for joint 1 coefficients
    B1 = [th1_i; v0; th1_f; vf];
    a1 = T \ B1;

    % Solve for joint 2 coefficients
    B2 = [th2_i; v0; th2_f; vf];
    a2 = T \ B2;

    % Evaluate theta1 and theta2 at time t
    % theta1 = a1(1) + a1(2)*t + a1(3)*t^2 + a1(4)*t^3;
    % theta2 = a2(1) + a2(2)*t + a2(3)*t^2 + a2(4)*t^3;
end
```

```
function [Th2, Th1 ]= return_angle(a1,  a2, tf, clock)

    t = clock;

    % Clamp time to [0, tf] for safety
    if t < 0
        t = 0;
    elseif t > tf
        t = tf;
    end

    % Evaluate theta1 and theta2 at time t
    Th1 = a1(1) + a1(2)*t + a1(3)*t^2 + a1(4)*t^3;
    Th2= a2(1) + a2(2)*t + a2(3)*t^2 + a2(4)*t^3;

    % Plotting
    % figure;
    % plot(t, Th1, 'r', 'LineWidth', 1.5); % Red line for Theta1
    % hold on;
    % plot(t, Th2, 'b', 'LineWidth', 1.5); % Blue line for Theta2
    % hold off;


end
```

```
function V = PID_calculator( theta_desired, theta_feedback)
    % Parameters
    Kp = 1.75;
    Ki = 0.0;
    Kd = 10;

    persistent integral_error prev_error
    if isempty(integral_error)
        integral_error = 0;
        prev_error = 0;
    end

    dt = 0.1;  % Same as Simulink step size
    error = theta_desired - theta_feedback;
    integral_error = integral_error + error * dt;
    derivative_error = (error - prev_error) / dt;
    prev_error = error;

    V = Kp*error + Ki*integral_error + Kd*derivative_error;
    % V = 100;
end
```

```
function V = PID_calculator( theta_desired, theta_feedback)
    % Parameters
    Kp = 1.75;
    Ki = 0.0;
    Kd = 10;

    persistent integral_error prev_error
    if isempty(integral_error)
        integral_error = 0;
        prev_error = 0;
    end

    dt = 0.1;  % Same as Simulink step size
    error = theta_desired - theta_feedback;
    integral_error = integral_error + error * dt;
    derivative_error = (error - prev_error) / dt;
    prev_error = error;

    V = Kp*error + Ki*integral_error + Kd*derivative_error;
    % V = 100;
end
```

```
function [angular_acceleration , angular_velocity,theta ]= Motor_Dynamic(T_load, V)



    % T_load = 0
    % Motor constants
    J = 0.824;
    B = 0.0079;
    K = 1.3908;
    R = 1.7;
    L = 0.0163;

    b = (R * J + B * L);
    c = (R * B + K^2);
    e = (L * J);
    d = T_load * R;

    persistent omega omega_dot theta_val
    if isempty(omega)
        omega = 0;
        omega_dot = 0;
        theta_val = 0;
    end

    dt = 0.01;
    omega_ddot = ((V*K - b * omega_dot - c * omega - d)/e);
    omega_dot = omega_dot + omega_ddot * dt;
    omega = omega + omega_dot * dt;
    theta_val = theta_val + omega * dt;

    theta = theta_val;
    angular_velocity = omega;
    angular_acceleration = omega_dot;

    % x(1) = theta;
    % x(2)= omega;
    % x(3)= omega_dot;
    %
    % xdot = [x(2);
    %     x(3);
    %     ((V*K - b * x(3) - c * x(2) - d)/e);];
end
```

```
function [theta, angular_velocity, angular_acceleration] = Motor_Dynamic(T_load, V)
% function [theta,angular_velocity,angular_acceleration] = Motor_Dynamic(T_load, V)
%     % T_load = 0
%     % Motor constants
%     J = 0.824;
%     B = 0.0079;
%     K = 1.3908;
%     R = 1.7;
%     L = 0.0163;
%
%     b = (R * J + B * L);
%     c = (R * B + K^2);
%     e = (L * J);
%     d = T_load * R;
%
%     persistent omega omega_dot theta_val
%     if isempty(omega)
%         omega = 0;
%         omega_dot = 0;
%         theta_val = 0;
%     end
%
%     dt = 0.01;
%     omega_ddot = ((V*K - b * omega_dot - c * omega - d)/e);
%     omega_dot = omega_dot + omega_ddot * dt;
%     omega = omega + omega_dot * dt;
%     theta_val = theta_val + omega * dt;
%
%     theta = theta_val;
%     angular_velocity = omega;
%     angular_acceleration = omega_dot;
%
%
%
% end


% function [angular_acceleration , angular_velocity,theta ]= Motor_Dynamic(T_load, V)



    % T_load = 0
    % Motor constants
    J = 0.824;
    B = 0.0079;
    K = 1.3908;
    R = 1.7;
    L = 0.0163;

    b = (R * J + B * L);
    c = (R * B + K^2);
    e = (L * J);
    d = T_load * R;

    persistent omega omega_dot theta_val
    if isempty(omega)
        omega = 0;
        omega_dot = 0;
        theta_val = 0;
    end
```

```
    dt = 0.01;
    omega_ddot = ((V*K - b * omega_dot - c * omega - d)/e);
    omega_dot = omega_dot + omega_ddot * dt;
    omega = omega + omega_dot * dt;
    theta_val = theta_val + omega * dt;

    theta = theta_val;
    angular_velocity = omega;
    angular_acceleration = omega_dot;

    % x(1) = theta;
    % x(2)= omega;
    % x(3)= omega_dot;
    %
    % xdot = [x(2);
    %     x(3);
    %     ((V*K - b * x(3) - c * x(2) - d)/e);];
end
```